PEMROGRAMAN BERORIENTASI OBJEK 2026

Pewarisan (*Inheritance*)

Pilar kedua PBO: bagian yang sama ditulis sekali di kelas induk, lalu diturunkan ke kelas anak

PERTEMUAN

07

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan masalah duplikasi kode dan cara **generalisasi** menyelesaikannya.
2. Menentukan apakah hubungan dua kelas termasuk *is-a* (pewarisan) atau *has-a* (relasi biasa).
3. Membuat kelas anak yang memanggil konstruktor induk dengan `super`, lalu menambah atribut dan *method* baru.
4. Meng-*override method* induk sambil tetap memakai versi induknya.
5. Menjelaskan urutan berjalannya konstruktor pada pewarisan bertingkat.
6. Membedakan aturan pewarisan di Python dan Java: hak akses untuk kelas anak, kelas akar, pewarisan ganda, dan `final`.

Mengingat Kembali



Dari pertemuan sebelumnya

- Pertemuan 05: kelas dihubungkan lewat asosiasi, agregasi, komposisi, dan dependensi, semuanya hubungan "**punya**" atau "**memakai**".
- Contohnya, `Kedai` menyimpan daftar `Menu` (agregasi) dan `Pesanan` menyusun `ItemPesanan` (komposisi).
- Pertemuan 06: notasi **generalisasi** berupa garis dengan segitiga kosong yang menunjuk ke kelas induk (`<|--`).
- `__str__`/`toString()` disebut "menimpa *method* bawaan" (P03), dan `protected` disebut sekilas (P04).

Agenda Hari Ini

1. Kode kembar dan generalisasi
2. Relasi “adalah” (*is-a*)
3. Konstruktor anak dan `super`
4. *Override*: menimpa *method* induk
5. Hak akses, kelas akar, dan *has-a*
6. Studi kasus KedaiKita

BAGIAN 1

1

Kode Kembar dan Generalisasi

Masalah yang muncul saat dua kelas ditulis hampir sama persis.

Masalah: `Makanan` dan `Minuman` yang Kembar

Makanan
-kode : String -nama : String -harga : int -stok : int -levelPedas : int
+kurangiStok(jumlah) : void +hargaSetelahDiskon(persen) : int +toString() : String

Minuman
-kode : String -nama : String -harga : int -stok : int -dingin : boolean
+kurangiStok(jumlah) : void +hargaSetelahDiskon(persen) : int +toString() : String

Hampir seluruh isi kedua kotak sama persis. Hanya satu atribut yang berbeda di masing-masing kelas.

Pewarisan: Kelas Induk dan Kelas Anak

Pewarisan (*inheritance*)

Mekanisme ketika sebuah kelas **mewarisi** atribut dan *method* dari kelas lain, lalu boleh **menambah** atau **mengubah** sebagiannya.

- Yang mewariskan disebut **kelas induk** (*superclass*); yang mewarisi disebut **kelas anak** (*subclass*).
- **Generalisasi**: mengumpulkan bagian yang sama ke induk. **Spesialisasi**: membuat anak yang lebih khusus.
- Sintaksnya: `class Makanan(Menu):` di Python, `class Makanan extends Menu` di Java.

BAGIAN 2

2

Relasi “Adalah” (*Is-a*)

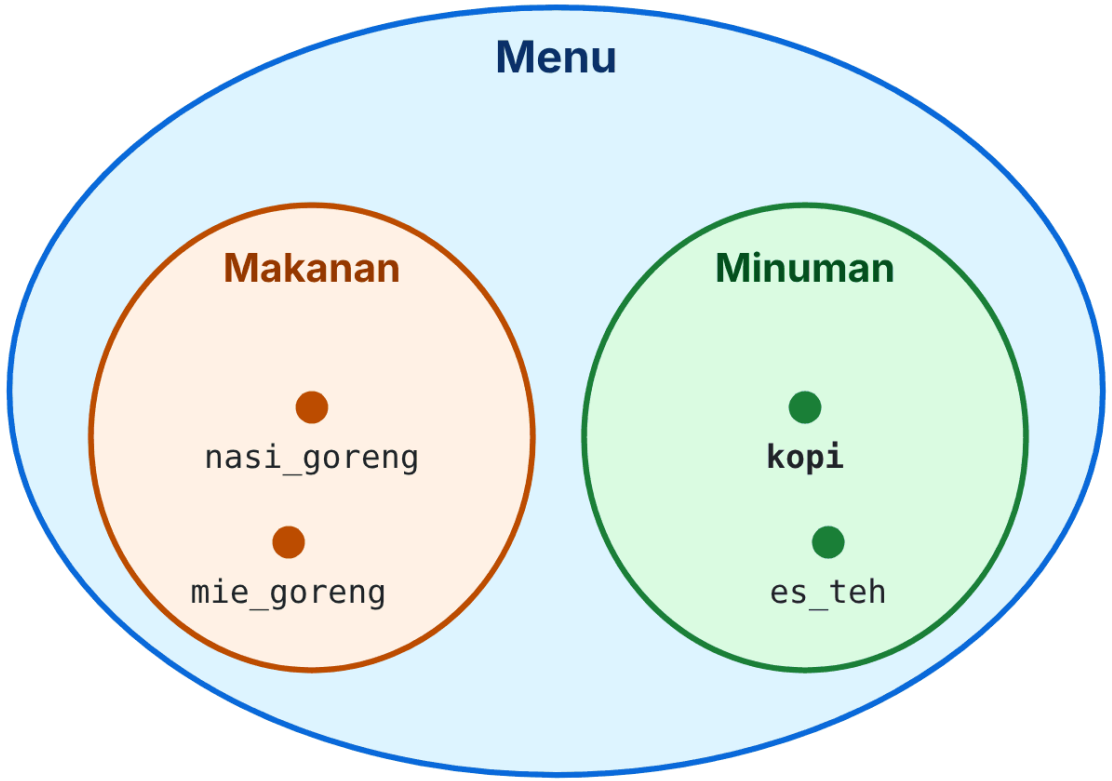
Kapan sebuah kelas boleh menjadi anak kelas lain?

Uji Kalimat “X Adalah Y”

Kalimat uji	Benar?	Kesimpulan
Makanan **adalah** Menu	ya	`Makanan` boleh menjadi anak `Menu`
Dosen **adalah** Pegawai	ya	`Dosen` boleh menjadi anak `Pegawai`
Laptop **adalah** Komputer	ya	`Laptop` boleh menjadi anak `Komputer`
Menu **adalah** Makanan	tidak	Arahnya terbalik: tidak semua menu adalah makanan
Mobil **adalah** Mesin	tidak	Mobil **punya** mesin (dibahas di bagian 5)
Pesanan **adalah** Pelanggan	tidak	Pesanan **milik** pelanggan (asosiasi, Pertemuan 05)

Jika kalimat “X adalah Y” benar untuk **setiap** X, pewarisan layak dipertimbangkan.

Kelas Anak Berada di dalam Kelas Induk



Tanya tentang objek kopi:

kopi adalah Minuman? **ya**

kopi adalah Menu? **ya**
(Minuman ada di dalam Menu)

kopi adalah Makanan? **tidak**

```
Python: isinstance(kopi, Menu)
Java : kopi instanceof Menu
```

Arahnya hanya ke atas: setiap Minuman adalah Menu, tetapi tidak setiap Menu adalah Minuman.

Setiap kelas adalah himpunan objek. Arahnya hanya ke atas: setiap `Minuman` adalah `Menu`, tetapi tidak setiap `Menu` adalah `Minuman`.

BAGIAN 3

3

Konstruktor Anak dan `super`

Siapa yang mengisi lapis induk saat objek anak dibuat?

Konstruktor Anak Memanggil `super`

`Komputer` mewarisi `Perangkat` (contoh 04). Baris pertama konstruktornya meminta induk mengisi lapis induk.

Python python/contoh/contoh04_urutan_konstruktor.py

```python
class Komputer(Perangkat):
    """Tingkat 2: Komputer adalah Perangkat."""

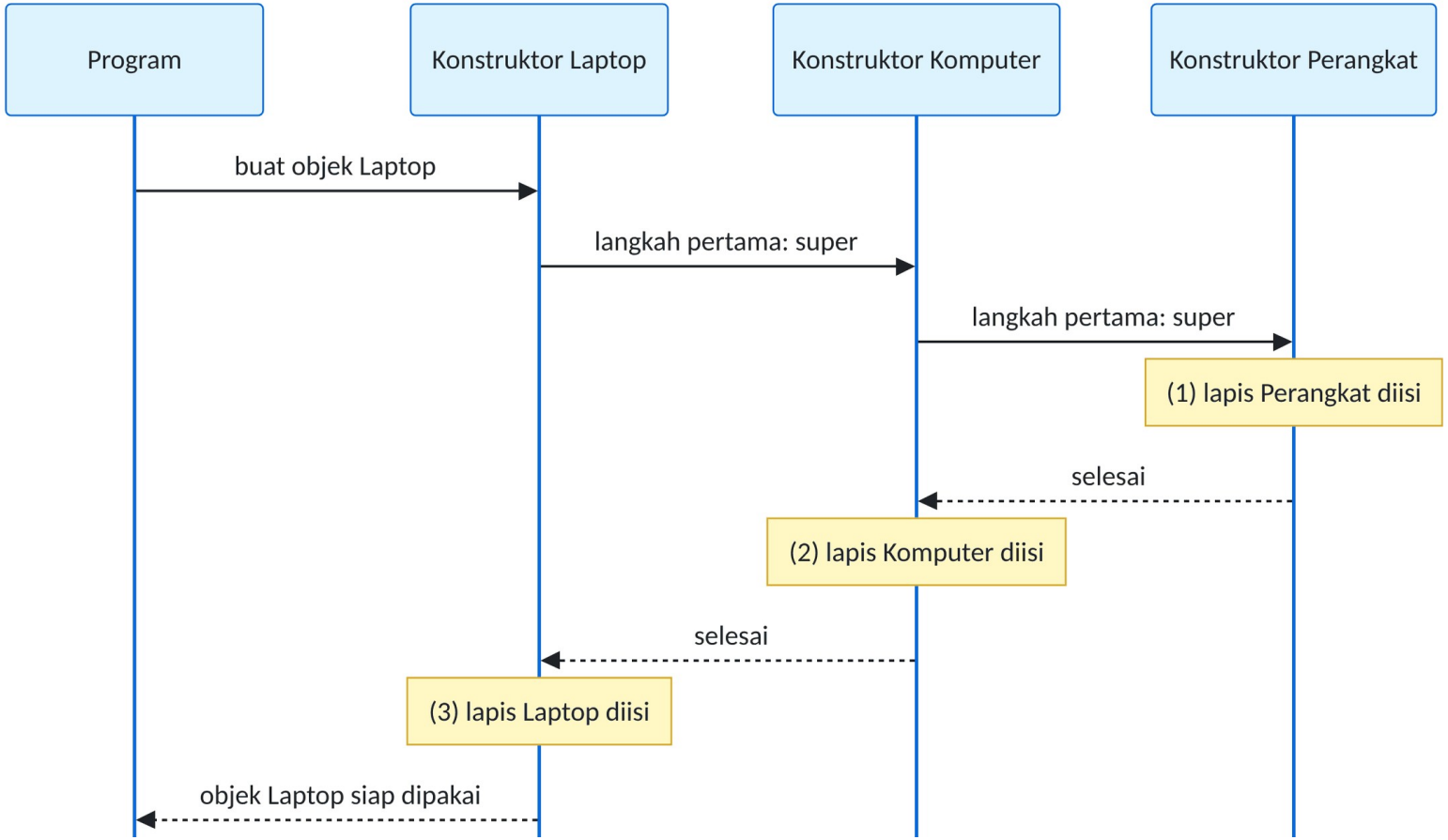
    def __init__(self, merek: str, ram_gb: int):
        super().__init__(merek)
        self._ram_gb = ram_gb
        print(f"  Bagian Komputer selesai: RAM {ram_gb} GB")
```

Java java/contoh/Contoh04UrutanKonstruktor.java

```java
/** Tingkat 2: Komputer adalah Perangkat. */
class Komputer extends Perangkat {
    private int ramGb;

    public Komputer(String merek, int ramGb) {
        super(merek);
        this.ramGb = ramGb;
        // ... (dilewati)
    }
    // ... (dilewati)
}
```

Urutan Konstruktor: Induk Selesai Lebih Dulu



Yang **dipanggil** pertama memang konstruktor anak, tetapi yang **selesai** pertama adalah lapis paling atas.

Aturan Konstruktor Anak di Java dan Python

Java

- Konstruktor **tidak** diwariskan; setiap kelas menentukan sendiri cara objeknya dibuat.
- `super(...)` wajib menjadi **pernyataan pertama** di konstruktor anak (Java 17 dan 21).
- Jika tidak ditulis, Java diam-diam memanggil `super()` tanpa argumen.
- Kompilasi gagal jika induk tidak punya konstruktor tanpa parameter.

Python

- `__init__` adalah *method* biasa, jadi ikut diwariskan jika anak tidak menulisnya.
- Tidak ada pemeriksaan otomatis; biasakan `super().__init__(...)` di **baris pertama**.
- Jika lupa ditulis, atribut induk tidak pernah dibuat.
- Program baru gagal (`AttributeError`) saat atribut itu dipakai.

BAGIAN 4

4

Override: Menimpa *Method* Induk

Kelas anak boleh mengganti perilaku warisan, khusus untuk dirinya.

Override dan Dua Gayanya

Override (menimpa)

Menulis ulang *method* milik kelas induk di kelas anak, dengan **nama dan parameter yang sama** sehingga objek kelas anak memakai versi barunya.

- **Mengganti total**: versi anak sama sekali tidak memakai versi induk.
- **Memperluas**: versi anak memanggil versi induk lewat `super`, lalu menambahkan sesuatu. Gaya ini paling sering dipakai.
- Java memakai anotasi `@Override` dan melarang hak akses yang lebih sempit; Python tidak memeriksa.

Override yang Memperluas Versi Induk

`TiketVIP` mewarisi `Tiket`, lalu menimpa dua *method*. Versi induk tetap dipakai lewat `super`.

Python python/contoh/contoh03_override.py

```python
class TiketVIP(Tiket):
    # ... (dilewati)
    def info(self) -> str:
        return f"{super().info()} | VIP: {self._fasilitas}"

    def hitung_bayar(self, jumlah: int) -> int:
        harga_dasar = super().hitung_bayar(jumlah)
        return harga_dasar + BIAYA_LAYANAN_VIP * jumlah
```

Java java/contoh/Contoh03Override.java

```java
class TiketVIP extends Tiket {
    // ... (dilewati)
    @Override
    public String info() {
        return super.info() + " | VIP: " + fasilitas;
    }

    @Override
    public int hitungBayar(int jumlah) {
        int hargaDasar = super.hitungBayar(jumlah);
        return hargaDasar + BIAYA_LAYANAN_VIP * jumlah;
    }
}
```

BAGIAN 5

5

Hak Akses, Kelas Akar, dan Kapan Tidak Mewarisi

Beda aturan Python dan Java, lalu jebakan desain yang paling sering terjadi.

Beda Aturan Pewarisan di Python dan Java

Konsep	Python	Java
Atribut untuk kelas anak	kesepakatan `_nama`	`protected` (juga terbuka untuk satu paket)
Tertutup bagi kelas anak	`__nama` (*name mangling*)	`private` (tetap ada di objek anak)
Kelas akar	`object`	`Object`
Teks objek bawaan yang di-*override*	`__str__`	`toString()`
Lebih dari satu induk	boleh; urutan pencariannya disebut MRO (*Method Resolution Order*)	tidak boleh; satu induk + *interface* (Pertemuan 10)
Mengunci kelas atau *method*	tidak ada kata kunci	`final`

Pilihan aman di kedua bahasa: buat atribut tertutup, lalu sediakan *method* resmi yang juga dipakai kelas anak.

Mobil Bukan Mesin: *Is-a* vs *Has-a*

Mobil butuh `hidupkan()` milik `Mesin`. Rancangan keliru mewarisi `Mesin`; rancangan tepat **menyimpan** objek `Mesin`.

Python python/contoh/contoh07_is_a_dan_has_a.py

```python
class MobilKeliru(Mesin):
    # ... (dilewati)

class MobilTepat:
    # ... (dilewati)
    def __init__(self, nama: str, mesin: Mesin):
        self._nama = nama
        self._mesin = mesin

    def jalan(self) -> str:
        return f"{self._nama}: {self._mesin.hidupkan()}"

    def ganti_mesin(self, mesin_baru: Mesin):
        self._mesin = mesin_baru
```

Java java/contoh/Contoh07IsADanHasA.java

```java
class MobilKeliru extends Mesin {
    // ... (dilewati)
}
class MobilTepat {
    private String nama;
    private Mesin mesin;
    // ... (dilewati)
    public String jalan() {
        return nama + ": " + mesin.hidupkan();
    }
    public void gantiMesin(Mesin mesinBaru) {
        this.mesin = mesinBaru;
    }
}
```

Is-a atau *Has-a*?

Aspek	*is-a* (pewarisan)	*has-a* (relasi Pertemuan 05)
Kalimat uji	"X **adalah** Y"	"X **punya** Y"
Contoh	Makanan adalah Menu	Pesanan punya ItemPesanan
Wujud di kode	kelas anak menyebut induknya	atribut berisi objek lain
Bisa berubah saat program berjalan?	tidak, tetap seumur hidup objek	bisa, misalnya mesin diganti
Simbol di diagram kelas	segitiga kosong	garis, panah, atau belah ketupat

Prinsip ***composition over inheritance***: warisi hanya jika hubungannya benar-benar *is-a*.

STUDI KASUS

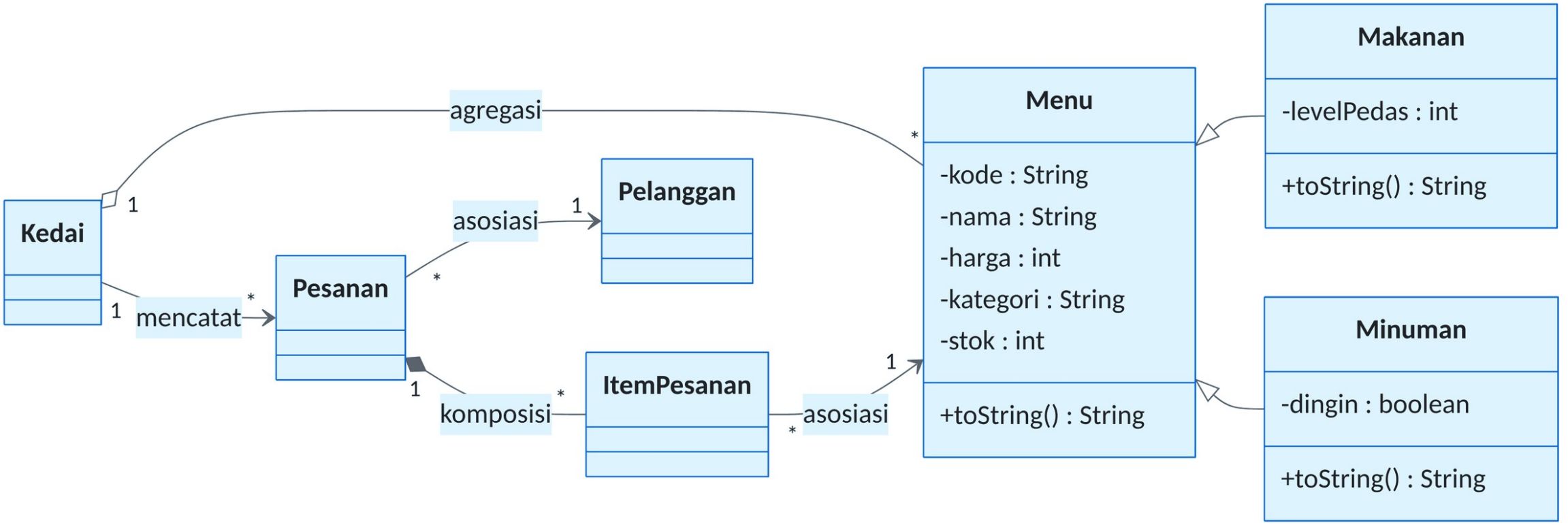


Studi Kasus KedaiKita

`Makanan` dan `Minuman` resmi menjadi kelas anak `Menu`.

Diagram Kelas KedaiKita Pertemuan 07

Studi kasus KedaiKita



Bagian yang baru hanya `Makanan`, `Minuman`, dan dua garis pewarisan. Kelas lain tidak disentuh sama sekali.

Kelas `Minuman` Mewarisi `Menu`

Studi kasus KedaiKita

Kepala konstruktor yang panjang dipersingkat dengan `...`. Kategori `"Minuman"` dikirim otomatis ke konstruktor induk.

Python python/studi-kasus/menu.py

```python
class Minuman(Menu):
    def __init__(self, kode: str, ..., dingin: bool = True):
        super().__init__(kode, nama, harga, "Minuman", stok)
        self._dingin = dingin
    # ... (dilewati)
    def __str__(self) -> str:
        suhu = "dingin" if self._dingin else "panas"
        return f"{super().__str__()} | {suhu}"
```

Java java/studi-kasus/Minuman.java

```java
public class Minuman extends Menu {
    private boolean dingin;
    public Minuman(String kode, ..., boolean dingin) {
        super(kode, nama, harga, "Minuman", stok);
        this.dingin = dingin;
    }
    // ... (dilewati)
    @Override
    public String toString() {
        String suhu = dingin ? "dingin" : "panas";
        return super.toString() + " | " + suhu;
    }
}
```

`Kedai` Tidak Diubah, Menu Tetap Tampil Lengkap

Studi kasus KedaiKita

`tambah_menu` menerima `Menu`, dan `tampilkan_menu` hanya mencetak setiap menu tanpa tahu jenisnya.

Python python/studi-kasus/kedai.py

```python
def tambah_menu(self, menu: Menu):
    self._daftar_menu.append(menu)
# ... (dilewati)
def tampilkan_menu(self):
    print(f"=== MENU {self._nama} ===")
    for menu in self._daftar_menu:
        print(menu)
```

Java java/studi-kasus/Kedai.java

```java
public void tambahMenu(Menu menu) {
    daftarMenu.add(menu);
}
// ... (dilewati)
public void tampilkanMenu() {
    System.out.println("=== MENU " + nama + " ===");
    for (Menu menu : daftarMenu) {
        System.out.println(menu);
    }
}
```

Output

```
=== MENU KedaiKita ===
[M01] Kopi Susu Gula Aren (Minuman) - Rp18.000 | stok 20 | dingin
[M02] Es Teh Manis (Minuman) - Rp8.000 | stok 30 | dingin
[M03] Nasi Goreng Kampung (Makanan) - Rp25.000 | stok 15 | pedas level 2
[M04] Mie Goreng Jawa (Makanan) - Rp23.000 | stok 10 | pedas level 1
[M05] Pisang Goreng Keju (Makanan) - Rp15.000 | stok 12 | pedas level 0
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Kalau dua kelas punya kode yang mirip, salah satunya harus mewarisi yang lain.”	Pewarisan hanya tepat untuk hubungan *is-a*. Untuk hubungan “punya”, pakai atribut (*has-a*). Kemiripan kode saja bukan alasan.
“Kelas anak itu seperti anak manusia: keturunan dari induknya.”	Seorang anak bukan jenis dari ayahnya, sedangkan `Makanan` adalah **jenis dari** `Menu`. Pegang kalimat “anak **adalah** induk”.
“*Override* dan *overload* itu sama saja.”	*Override* mengganti *method* induk dengan nama dan parameter yang sama. *Overload* membuat variasi *method* dengan parameter berbeda.

Kuis Cepat

Pasangan manakah yang paling tepat dimodelkan dengan pewarisan, dengan kelas **pertama** sebagai kelas anak dan kelas **kedua** sebagai kelas induk?

A `Mobil` dan `Mesin`

B `Pesanan` dan `ItemPesanan`

C `Dosen` dan `Pegawai`

D `Kedai` dan `Menu`

Kuis Cepat: Jawaban

Pasangan manakah yang paling tepat dimodelkan dengan pewarisan, dengan kelas **pertama** sebagai kelas anak dan kelas **kedua** sebagai kelas induk?

A `Mobil` dan `Mesin`

B `Pesanan` dan `ItemPesanan`

✓ `Dosen` dan `Pegawai`

D `Kedai` dan `Menu`

Jawaban: C

Uji dengan kalimat "X **adalah** Y". "Dosen adalah pegawai" benar, jadi hubungannya *is-a*. Tiga pilihan lain adalah *has-a*: mobil **punya** mesin, pesanan **punya** item pesanan (komposisi), dan kedai **punya** daftar menu (agregasi). Hubungan "punya" diwujudkan dengan atribut, bukan pewarisan.

Kuis Cepat

Di Java, kelas `Kendaraan` hanya punya satu konstruktor, yaitu `Kendaraan(String merek)`. Konstruktor kelas anak `Motor` **tidak** menulis `super(...)` sama sekali. Apa yang terjadi?

A Java menyisipkan `super()` tanpa argumen secara otomatis sehingga kompilasi gagal

B Program berjalan normal, dan *field* `merek` bernilai `null`

C Java otomatis meneruskan semua parameter `Motor` ke konstruktor `Kendaraan`

D Konstruktor `Kendaraan` dilewati, hanya konstruktor `Motor` yang berjalan

Kuis Cepat: Jawaban

Di Java, kelas `Kendaraan` hanya punya satu konstruktor, yaitu `Kendaraan(String merek)`. Konstruktor kelas anak `Motor` **tidak** menulis `super(...)` sama sekali. Apa yang terjadi?

✓ Java menyisipkan `super()` tanpa argumen secara otomatis sehingga kompilasi gagal

B. Program berjalan normal, dan *field* `merek` bernilai `null`

C. Java otomatis meneruskan semua parameter `Motor` ke konstruktor `Kendaraan`

D. Konstruktor `Kendaraan` dilewati, hanya konstruktor `Motor` yang berjalan

Jawaban: A

Java menyisipkan `super();` di baris pertama, dan kompilasi gagal karena `Kendaraan` tidak punya konstruktor tanpa parameter. Java tidak pernah melewati konstruktor induk (D) dan tidak menebak argumen (C). Pilihan B mirip yang terjadi di Python jika lupa `super().__init__(...)`: program berjalan, lalu gagal belakangan.

Ringkasan

- Pewarisan menghilangkan **duplikasi**: bagian yang sama ditulis sekali di **kelas induk**, anak menulis bagian khususnya.
- Pewarisan adalah relasi ***is-a***, diuji dengan kalimat “X adalah Y”. Objek anak juga terhitung objek induk.
- Kelas anak mewarisi atribut dan *method* induk, lalu boleh menambah yang baru.
- Konstruktor anak memanggil konstruktor induk dengan `super`, dan lapis induk selalu **selesai lebih dulu**.
- ***Override*** menimpa *method* induk dengan nama dan parameter sama; versi induk tetap dipakai lewat `super`.
- Java `protected` terbuka untuk anak **dan** satu paket; Python memakai `_nama`. Java hanya boleh satu induk.
- Untuk hubungan “punya”, pakai ***has-a***. Prinsipnya: *composition over inheritance*.

Langkah Berikutnya

- Jalankan contoh 01—08 (Python) atau 01—07 (Java) dan studi kasus KedaiKita di jalur pilihanmu.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal.
- Lanjutkan ke Pertemuan 08 (UTS) dan kerjakan latihan soalnya.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-07-Pewarisan